# LAPORAN PRATIKUM DATA ENGINEERING AND BIG DATA SYSTEMS

**PERTEMUAN 4**


**OLEH**

**Nama : AYUL HUSNAINI**

**Nim : 105841110423**

**Kelas : 5 AI-A**

**PROGRAM STUDI INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MUHAMMADIYAH MAKASSAR**


**2026**

1. Import Library & Load Data Wine

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Memuat dataset anggur yang berisi 13 fitur kimia    "Memuat": Unknown word.
data_anggur = load_wine()    "anggur": Unknown word.
X_wine, y_wine = data_anggur.data, data_anggur.target    "anggur": Unknown word.

# Membagi data untuk latihan dan ujian (70:30)    "Membagi": Unknown word.
X_latih, X_uji, y_latih, y_uji = train_test_split(    "latih": Unknown word.
    X_wine, y_wine, test_size=0.3, random_state=42
)

# Inisialisasi model LogReg dengan iterasi tinggi agar konvergen sempurna    "Inisialisasi":
model_logreg = LogisticRegression(max_iter=5000).fit(X_latih, y_latih)    "logreg": Unknown
tebakan_logreg = model_logreg.predict(X_uji)    "tebakan": Unknown word.

# Inisialisasi model Decision Tree dengan batasan kedalaman 3 tingkat    "Inisialisasi": Unk
model_tree = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_latih, y_latih)    "
tebakan_tree = model_tree.predict(X_uji)    "tebakan": Unknown word.

print("Skor Akurasi Regresi Logistik :", round(accuracy_score(y_uji, tebakan_logreg), 3))
print("Skor Akurasi Pohon Keputusan  :", round(accuracy_score(y_uji, tebakan_tree), 3))    "
```

```
Skor Akurasi Regresi Logistik : 1.0
Skor Akurasi Pohon Keputusan  : 0.963
```

- from sklearn.datasets import load_wine

  Digunakan untuk:

  Memanggil fungsi **load_wine** dari scikit-learn untuk mengambil dataset anggur (wine) yang berisi fitur kimia dan label kelas.

- from sklearn.model_selection import train_test_split

  Digunakan untuk:

  Memanggil fungsi **train_test_split** untuk membagi data wine menjadi data latih dan data uji.

- from sklearn.linear_model import LogisticRegression

  Digunakan untuk:

  Memanggil kelas **LogisticRegression** untuk membuat dan melatih model klasifikasi pada data wine.

- from sklearn.tree import DecisionTreeClassifier

Digunakan untuk:

Memanggil kelas **DecisionTreeClassifier** untuk membuat dan melatih model pohon keputusan pada data wine.

- from sklearn.metrics import accuracy_score

  Digunakan untuk:

  Memanggil fungsi **accuracy_score** untuk menghitung persentase prediksi yang benar pada data uji.

- data_anggur = load_wine()

  X_wine, y_wine = data_anggur.data, data_anggur.target

  Digunakan untuk:

  Memuat data fitur dan label kelas dari dataset wine ke variabel X_wine dan y_wine.

- X_latih, X_uji, y_latih, y_uji = train_test_split(

  X_wine, y_wine, test_size=0.3, random_state=42)

  Digunakan untuk:

  Membagi data wine menjadi data latih (70%) dan data uji (30%) agar evaluasi model lebih menantang.

- model_logreg = LogisticRegression(max_iter=5000).fit(X_latih, y_latih) tebakan_logreg = model_logreg.predict(X_uji)

  Digunakan untuk:

  Membuat dan melatih model **Logistic Regression** dengan iterasi tinggi (5000) agar proses training pada data kimia yang kompleks berjalan optimal, lalu melakukan prediksi pada data uji.

- model_tree = DecisionTreeClassifier(max_depth=3,

  random_state=42).fit(X_latih, y_latih)tebakan_tree = model_tree.predict(X_uji)

  Digunakan untuk:

  Membuat dan melatih model **Decision Tree** dengan kedalaman maksimal 3, lalu melakukan prediksi pada data uji.

- print("Skor Akurasi Regresi Logistik :", round(accuracy_score(y_uji, tebakan_logreg), 3))

print("Skor Akurasi Pohon Keputusan :", round(accuracy_score(y_uji, tebakan_tree), 3))

Digunakan untuk:

Menampilkan skor akurasi dari kedua model pada data uji untuk membandingkan performa awal.

Output menampilkan skor akurasi LR sebesar 1.0, yang mengartikan model ini berhasil menebak seluruh data pengujian tanpa ada satu botol anggur pun yang salah golongan. Sementara itu, skor akurasi DT berhenti di angka 0.963. Hasil sekitar 96 persen ini bermakna bahwa aturan tiga cabang yang dipasang sebenarnya sudah sangat mumpuni untuk memilah ciri kimia mayoritas anggur, meskipun masih ada sedikit celah kesalahan.

2. perbandingkan metrik: accuracy, precision, recall, F1.

```
from sklearn.metrics import classification_report

# Menampilkan laporan statistik terperinci untuk model regresi logistik    "Menampilkan": Unknown word.
print("Laporan Performa Statistik: Regresi Logistik")    "Laporan": Unknown word.
print(classification_report(y_uji, tebakan_logreg, target_names=data_anggur.target_names))    "tebakan": Unknown word.

# Menampilkan laporan statistik terperinci untuk model pohon keputusan    "Menampilkan": Unknown word.
print("Laporan Performa Statistik: Pohon Keputusan")    "Laporan": Unknown word.
print(classification_report(y_uji, tebakan_tree, target_names=data_anggur.target_names))    "tebakan": Unknown word.
```

```
Laporan Performa Statistik: Regresi Logistik
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        19
     class_1       1.00      1.00      1.00        21
     class_2       1.00      1.00      1.00        14

    accuracy                           1.00        54
   macro avg       1.00      1.00      1.00        54
weighted avg       1.00      1.00      1.00        54

Laporan Performa Statistik: Pohon Keputusan
              precision    recall  f1-score   support

     class_0       1.00      0.95      0.97        19
     class_1       0.91      1.00      0.95        21
     class_2       1.00      0.93      0.96        14

    accuracy                           0.96        54
   macro avg       0.97      0.96      0.96        54
weighted avg       0.97      0.96      0.96        54
```

- from sklearn.metrics import classification_report

  Digunakan untuk:

  Memanggil fungsi **classification_report** untuk menampilkan metrik evaluasi detail (precision, recall, F1-score) tiap kelas.

- print("Laporan Performa Statistik: Regresi Logistik")

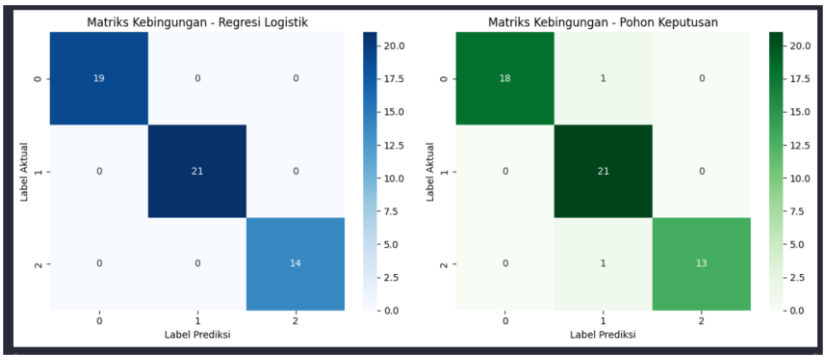
print(classification_report(y_uji,tebakan_logreg,
target_names=data_anggur.target_names))
Digunakan untuk:
Menampilkan laporan statistik lengkap untuk model Logistic Regression pada data uji.

- print("Laporan Performa Statistik: Pohon Keputusan")
print(classification_report(y_uji, tebakan_tree,
target_names=data_anggur.target_names))
Digunakan untuk:
Menampilkan laporan statistik lengkap untuk model Decision Tree pada data uji.

Output menampilkan sebaran nilai untuk tiap kategori anggur. Pada laporan regresi logistik, nilai presisi dan recall mendapatkan skor utuh 1.00 di semua kelas. Hal ini sejalan dengan nilai akurasi sebelumnya, menegaskan tidak ada tebakan yang tertukar sama sekali. Pada laporan pohon keputusan, terlihat ada penurunan nilai presisi di baris class_1 menjadi 0.91. Angka ini punya arti bahwa kalau pohon keputusan menebak sebuah anggur itu masuk ke class_1, ada peluang 9 persen kalau tebakan itu keliru. Lalu di bagian class_0 dan class_2, nilai recall berada di 0.95 dan 0.93; ini mengartikan ada segelintir data dari kedua kelas tersebut yang lolos dari deteksi model dan nyasar ke kelas lainnya. Meski begitu, nilai gabungan atau f1-score masih bertengger di angka 0.96, tanda bahwa mesin ini masih bekerja dengan sangat baik secara umum.

3. Confusion Matrix Visualisasi

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Menghitung matriks kebingungan    "Menghitung": Unknown word.
cm_logreg = confusion_matrix(y_uji, tebakan_logreg)    "logreg": Unknown word.
cm_tree = confusion_matrix(y_uji, tebakan_tree)    "tebakan": Unknown word.

# Membuat kanvas dengan 2 grafik bersebelahan (1 baris, 2 kolom)    "Membuat": Unknown word.
fig, axes = plt.subplots(1, 2, figsize=(12, 5))    "figsize": Unknown word.

# Plot Matriks Kebingungan 1: Regresi Logistik    "Matriks": Unknown word.
sns.heatmap(cm_logreg, annot=True, fmt='d', cmap='Blues', ax=axes[0])    "logreg": Unknown word.
axes[0].set_title('Matriks Kebingungan - Regresi Logistik')    "Matriks": Unknown word.
axes[0].set_xlabel('Label Prediksi')    "xlabel": Unknown word.
axes[0].set_ylabel('Label Aktual')    "ylabel": Unknown word.

# Plot Matriks Kebingungan 2: Pohon Keputusan    "Matriks": Unknown word.
sns.heatmap(cm_tree, annot=True, fmt='d', cmap='Greens', ax=axes[1])    "annot": Unknown word.
axes[1].set_title('Matriks Kebingungan - Pohon Keputusan')    "Matriks": Unknown word.
axes[1].set_xlabel('Label Prediksi')    "xlabel": Unknown word.
axes[1].set_ylabel('Label Aktual')    "ylabel": Unknown word.

# Menyesuaikan tata letak agar rapi dan menampilkannya    "Menyesuaikan": Unknown word.
plt.tight_layout()
plt.show()
```




- import matplotlib.pyplot as plt

  import seaborn as sns

  from sklearn.metrics import confusion_matrix

  Digunakan untuk:

  Memanggil library Matplotlib dan Seaborn untuk visualisasi, serta fungsi **confusion_matrix** untuk membuat matriks kebingungan.

- cm_logreg = confusion_matrix(y_uji, tebakan_logreg)

  cm_tree = confusion_matrix(y_uji, tebakan_tree)

  Digunakan untuk:

  Menghitung matriks kebingungan untuk kedua model pada data uji.

- fig, axes = plt.subplots(1, 2, figsize=(12, 5))

  Digunakan untuk:

  Membuat kanvas grafik dengan 2 subplot berdampingan.

- sns.heatmap(cm_logreg, annot=True, fmt='d', cmap='Blues', ax=axes[0])

  axes[0].set_title('Matriks Kebingungan - Regresi Logistik')

  axes[0].set_xlabel('Label Prediksi')

  axes[0].set_ylabel('Label Aktual')

  Digunakan untuk:

  Menampilkan matriks kebingungan model Logistic Regression dalam bentuk heatmap.

- sns.heatmap(cm_tree, annot=True, fmt='d', cmap='Greens', ax=axes[1])

  axes[1].set_title('Matriks Kebingungan - Pohon Keputusan')

  axes[1].set_xlabel('Label Prediksi')

  axes[1].set_ylabel('Label Aktual')

Digunakan untuk:

Menampilkan matriks kebingungan model Decision Tree dalam bentuk heatmap.

- plt.tight_layout()

  plt.show()

  Digunakan untuk:

  Merapikan tata letak grafik dan menampilkan kedua heatmap ke layar.

4. Perbandingan model

```
# Perhitungan evaluasi akhir untuk menentukan model terbaik    "Perhitungan": Unknown word.
skor_final_lr = accuracy_score(y_uji, tebakan_logreg)    "skor": Unknown word.
skor_final_dt = accuracy_score(y_uji, tebakan_tree)    "skor": Unknown word.

print("HASIL EVALUASI AKHIR MODEL")    "HASIL": Unknown word.
print(f"Skor Akurasi Akhir LogReg : {skor_final_lr:.4f}")    "Skor": Unknown word.
print(f"Skor Akurasi Akhir Tree   : {skor_final_dt:.4f}")    "Skor": Unknown word.

if skor_final_lr > skor_final_dt:    "skor": Unknown word.
    print("Kesimpulan: Algoritma Logistic Regression dipilih sebagai model terbaik.")    "Kesimpulan": Unknown word.
    print("Alasan: Memberikan hasil prediksi yang lebih akurat pada fitur kimia dataset wine.")    "Alasan": Unknown
else:
    print("Kesimpulan: Algoritma Decision Tree dipilih sebagai model terbaik.")    "Kesimpulan": Unknown word.
    print("Alasan: Mampu menangkap pola percabangan data yang lebih efektif pada pengujian ini.")    "Alasan": Unkno

HASIL EVALUASI AKHIR MODEL
Skor Akurasi Akhir LogReg : 1.0000
Skor Akurasi Akhir Tree   : 0.9630
Kesimpulan: Algoritma Logistic Regression dipilih sebagai model terbaik.
Alasan: Memberikan hasil prediksi yang lebih akurat pada fitur kimia dataset wine.
```

Langkah awal dilakukan dengan cara menghitung ulang angka akurasi mutlak dari kedua model. Fungsi `accuracy_score` kembali bertugas menghitungnya, lalu hasilnya disimpan ke variabel `skor_final_lr` untuk regresi logistik dan `skor_final_dt` untuk pohon keputusan. Teks pembuka dicetak, dilanjutkan dengan mencetak skor kedua model tersebut. Sisipan teks `:.4f` diletakkan di kurung kurawal agar angka desimalnya dipotong sampai empat digit saja biar rapi.

Sebuah blok persyaratan `if-else` kemudian dibangun untuk membandingkan kedua skor. Kalau nilai di `skor_final_lr` lebih besar, maka program akan masuk ke blok `if` dan mencetak kalimat penobatan kemenangan untuk regresi logistik. Sebaliknya, kalau nilai pohon keputusan yang menang atau seimbang, program akan masuk ke jalur `else` dan mencetak teks kesimpulan alternatifnya.

Teks yang keluar menunjukkan regresi logistik terpilih sebagai model yang lebih baik pada pengujian ini karena nilai skor 1.000 lebih besar dari 0.9630. Alasan utamanya berhubungan dengan isi datanya. Dataset anggur memiliki 13 ciri kimia yang berbentuk angka bervariasi. Regresi logistik bekerja lebih baik karena hitungan peluangnya lebih cocok memproses angka-angka yang bentuknya berkelanjutan seperti ini. Sedangkan pembatasan aturan pada pohon keputusan justru membuat proses pilah datanya menjadi terlalu kaku, sehingga beberapa tebakan akhirnya masuk ke kelompok yang salah.

5. Kesimpulan

Praktikum pertemuan keempat ini memberikan pemahaman langsung tentang cara algoritma mengelompokkan data. Algoritma yang dicoba ternyata mempunyai kecocokan yang berbeda pada setiap bentuk data. Regresi logistik bekerja sangat baik pada data angka yang ciri kimianya beragam, asalkan pengaturan batas hitungannya diperbesar agar pencariannya tidak terhenti. Di sisi lain, pohon keputusan punya cara pemilahan yang lebih mudah dimengerti, namun batas aturan cabangnya kadang menyebabkan ada data yang meleset. Pemakaian alat ukur seperti matriks kebingungan dan laporan klasifikasi juga menjadi keharusan, karena persentase akurasi saja tidak bisa memperlihatkan bagian mana saja yang ditebak dengan salah oleh model.